

Multiple Linear Regression for Baseball Win Prediction

Homework #1

Naomi Buell Richie Rivera Alexander Simon Nana Frimpong
Said Naqwe

2026-03-01

1 Introduction

We analyze and model data on a professional baseball team from 1871 to 2006. Each record has the performance of the team for the given year, with all of the statistics adjusted to match the performance of a 162 game season. We build a multiple linear regression model on the training data to predict the number of wins for the team.

2 Methods

2.1 Data Exploration

Describe the size and the variables in the moneyball training data set.

2.1.1 Data Characteristics

The moneyball training data set contains 2276 rows and 17 features, and is 91.01 percent complete. The data are all numeric, and the target variable is `TARGET_WINS`, which represents the number of games won by the team. The other variables represent various performance statistics for the team. See Table 1 for descriptive characteristics of the training data. Variables range from 0% to 92% missing. We address missingness by dropping columns and imputing in the Data Preparation section.

Table 1: Summary statistics for all variables in the training dataset

Variable	Completeness rate	Mean	Min	P25	P50	P75	Max
INDEX	1.000	1300	1	630	1300	1900	2500
TARGET_WINS	1.000	81	0	71	82	92	150
TEAM_BAT- TING_H	1.000	1500	890	1400	1500	1500	2600

Table 1: Summary statistics for all variables in the training dataset

Variable	Completeness rate	Mean	Min	P25	P50	P75	Max
TEAM_BAT- TING_2B	1.000	240	69	210	240	270	460
TEAM_BAT- TING_3B	1.000	55	0	34	47	72	220
TEAM_BAT- TING_HR	1.000	100	0	42	100	150	260
TEAM_BAT- TING_BB	1.000	500	0	450	510	580	880
TEAM_BAT- TING_SO	0.960	740	0	550	750	930	1400
TEAM_BASERUN_SB	0.940	120	0	66	100	160	700
TEAM_BASERUN_CS	0.660	53	0	38	49	62	200
TEAM_BAT- TING_HBP	0.084	59	29	50	58	67	95
TEAM_PITCH- ING_H	1.000	1800	1100	1400	1500	1700	30000
TEAM_PITCH- ING_HR	1.000	110	0	50	110	150	340
TEAM_PITCH- ING_BB	1.000	550	0	480	540	610	3600
TEAM_PITCH- ING_SO	0.960	820	0	620	810	970	19000
TEAM_FIELD- ING_E	1.000	250	65	130	160	250	1900
TEAM_FIELD- ING_DP	0.870	150	52	130	150	160	230

2.1.2 Correlation Matrix

The correlation between all variables are shown Figure 1. The most correlated variables with the target variable `TARGET_WINS` are `TEAM_BATTING_H`, `TEAM_BATTING_2B`, and `TEAM_BATTING_BB`, which may be predictors of wins. The most correlated independent variables are `TEAM_BATTING_HR` and `TEAM_PITCHING_HR`, which makes sense as teams that hit many homeruns may also allow many homeruns, for example.

Figure 2 shows the distribution of z-scores of all variable observations, and identifies outliers in the training data. `TEAM_PITCHING_SO`, `TEAM_PITCHING_H`, and `TEAM_PITCHING_BB` have many high

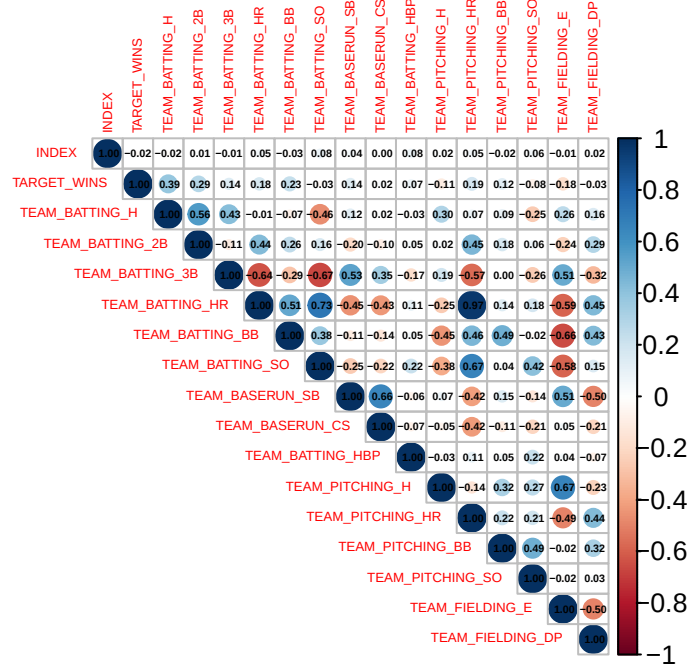


Figure 1: Correlation matrix of all variables in the training dataset

outliers. If there are bad leverage points in our models, we may need to create dummy variables for the outliers.

2.2 Data Preparation

We handled high missingness in the variables `TEAM_BATTING_HBP`, `TEAM_BASERUN_CS`, and `TEAM_FIELDING_DP` (91.6%, 33.9%, and 12.6% missing, respectively), which are not very correlated with the target variable, by excluding these variables. The remaining variables had less than 10% missing, so we will impute the missing values with the median of each variable. We avoid dropping `TEAM_BASERUN_SB` as it is the most correlated variable with the target variable that has missingness, and instead impute it.

The median imputation was picked based on the distributions of missing variables: `TEAM_BASERUN_SB` and `TEAM_PITCHING_SO` have a long right tail so the median is a better imputation method than the mean. Both variables have observed have extreme outliers pulling the mean towards a right direction compared to where most the data sits. When the distribution of variables reveals two distinct peaks as observed in `TEAM_BATTING_SO`, we can refer to this distribution as bimodal. Since the mean of bimodal falls between two peaks, this area is observed to be the underrepresented region of the missing data. The median imputation is utilized to approach one of the two clusters of missing variables with the largest distribution of missing variables. Below are the histograms for the distributions of these variables.

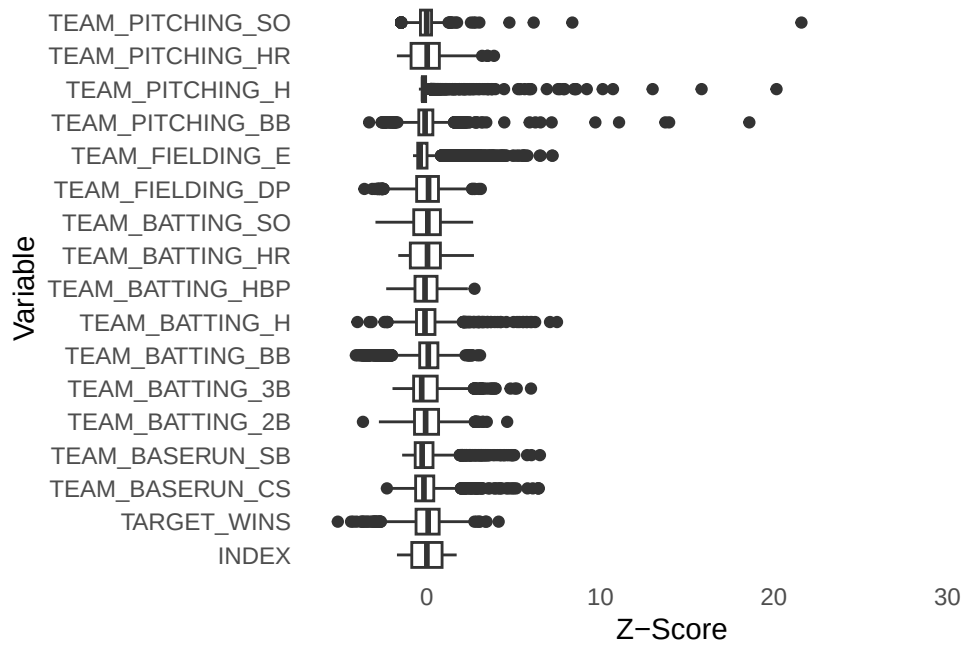


Figure 2: Box plots for identifying outliers in the training data

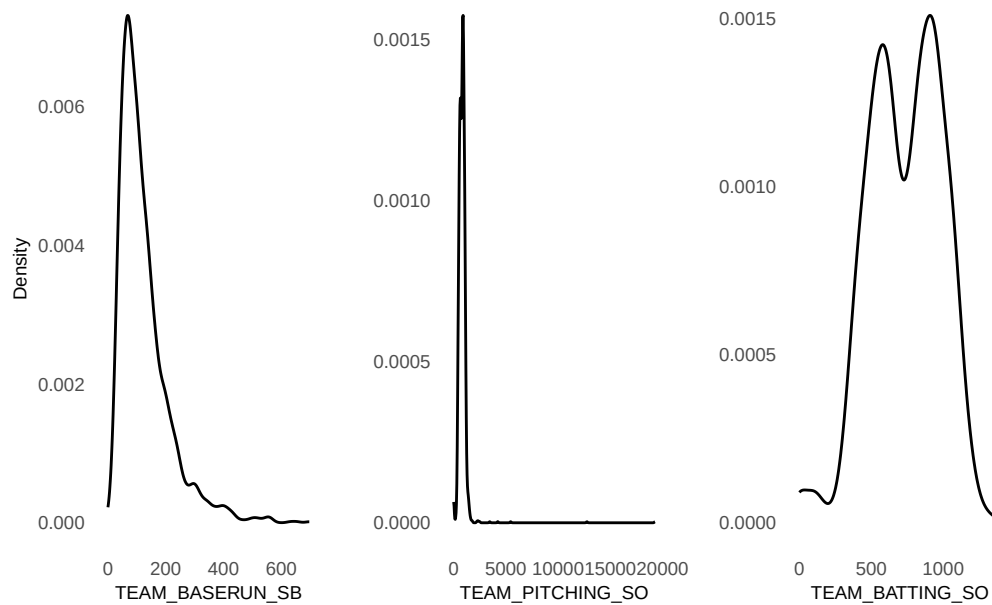


Figure 3: Histograms of variables with missingness selected for imputation

To begin by remove variables with high missingness variables and low correlation with **TARGET_WINS**: **TEAM_BATTING_HBP** (91.6% missing, $r=0.07$), **TEAM_BASERUN_CS** (33.9% missing, $r=0.02$), and **TEAM_FIELDING_DP** (12.6% missing, $r=-0.03$).

```
# Drop high missingness variables and low-correlation variables with the data
# These variables lack a strong relationship with response variable TARGET_WINS
moneyball_clean <- moneyball_train |>
  select(-TEAM_BATTING_HBP, -TEAM_BASERUN_CS, -TEAM_FIELDING_DP)
```

Before our predictor variables TEAM_BASERUN_CS, TEAM_BASERUN_SB and TEAM_PITCHING_SO are imputed, a binary indicator is created to preserve information that the value was originally missing. Our indicator for each flag is (1 = missing, 0 = present)

```
moneyball_clean <- moneyball_clean |>
  mutate(
    TEAM_BASERUN_SB_MISSING = as.integer(is.na(TEAM_BASERUN_SB)),
    TEAM_BATTING_SO_MISSING = as.integer(is.na(TEAM_BATTING_SO)),
    TEAM_PITCHING_SO_MISSING = as.integer(is.na(TEAM_PITCHING_SO))
  )
```

Imputation is now based on the distribution of missing data of the following variables. TEAM_BASERUN_SB, TEAM_PITCHING_SO, and TEAM_BATTING_SO. TEAM_BASERUN_SB and TEAM_PITCHING_SO are right skewed, therefore they are imputed around the mean. TEAM_BATTING_SO is bimodal, so we can impute around the median to avoid placing the imputed values in region between two peaks where only a few observations exist.

```
moneyball_clean <- moneyball_clean %>%
  mutate(
    # Mean imputation for right-tailed variables
    TEAM_BASERUN_SB = ifelse(
      is.na(TEAM_BASERUN_SB),
      mean(TEAM_BASERUN_SB, na.rm = TRUE),
      TEAM_BASERUN_SB
    ),
    TEAM_PITCHING_SO = ifelse(
      is.na(TEAM_PITCHING_SO),
      mean(TEAM_PITCHING_SO, na.rm = TRUE),
      TEAM_PITCHING_SO
    ),

    # Median imputation for bimodal variable
    TEAM_BATTING_SO = ifelse(
      is.na(TEAM_BATTING_SO),
      median(TEAM_BATTING_SO, na.rm = TRUE),
      TEAM_BATTING_SO
    )
  )
```

```
)
)
```

When a distribution is observed to be skewed to the right, meaning small number of outliers has influence on the response variable `TARGET_WINS`, the log transformation can be applied to balance the scale of distribution. Utilizing log compresses the distribution of missing variable closer to symmetric shape. This makes the relationship with `TARGET_WINS` more linear and easier for the model to capture. `log1p` is used to handle any zero without producing undefined results.

```
# Log transform right-skewed variables
moneyball_clean <- moneyball_clean %>%
  mutate(
    LOG_TEAM_BASERUN_SB = log1p(Team_Baserun_SB),
    LOG_TEAM_PITCHING_SO = log1p(Team_Pitching_SO),
    LOG_TEAM_FIELDING_E = log1p(Team_Fielding_E)
  )
```

We can create three new variables that captures negative and positive impact on the wins for the team.

```
# HR_DIFFERENTIAL and BAT_BB_SO_RATIO
moneyball_clean <- moneyball_clean |>
  mutate(
    # Net home run differential (offensive power vs pitching vulnerability)
    # The team that hits more home runs than it gives up will produce positive value
    # the team that allows more home runs than it hits will produce a negative
    # value, meaning they losing the game
    # Zero indicates the team is even in home runs
    HR_DIFFERENTIAL = Team_Batting_HR - Team_Pitching_HR,
    # Here we can measure how batter can judge which pitch they would take a swing at
    # If the ratio is greater than 1, it reveals the team walk more than they strike
    # showing a more cohesive and patient team
    # This may happen if a batter force the pitchers to throw more pitches
    BAT_BB_SO_RATIO = ifelse(Team_Batting_SO > 0,
                              Team_Batting_BB / Team_Batting_SO, NA)
  )

# Impute missing values in BAT_BB_SO_RATIO with median
moneyball_clean <- moneyball_clean |>
  mutate(
```

Table 2: Remaining missing values after imputation

```
# Verify that no variables have remaining missingness
moneyball_clean |>
  summarise(across(everything(), ~ sum(is.na(.)))) |>
  tidyr::pivot_longer(everything(),
                      names_to = "Variable",
                      values_to = "Missing") |>
  filter(Missing > 0) |>
  nrow()

[1] 0

# knitr::kable()
```

```
BAT_BB_SO_RATIO = ifelse(is.na(BAT_BB_SO_RATIO),
                        median(BAT_BB_SO_RATIO, na.rm = TRUE), BAT_BB_SO_RATIO)
)
```

Finally we can verify that there no missing variables that remain based on the results of our imputation

i Note

Comment from Alex: I tweaked the block below. The original code results in a blank output. While technically correct, it could make the prof wonder whether the code runs correctly. It'd be better to show some type of output, eg zero variables have missingness.

2.3 Assumptions for Linear Regression

Linear regression makes four important assumptions:

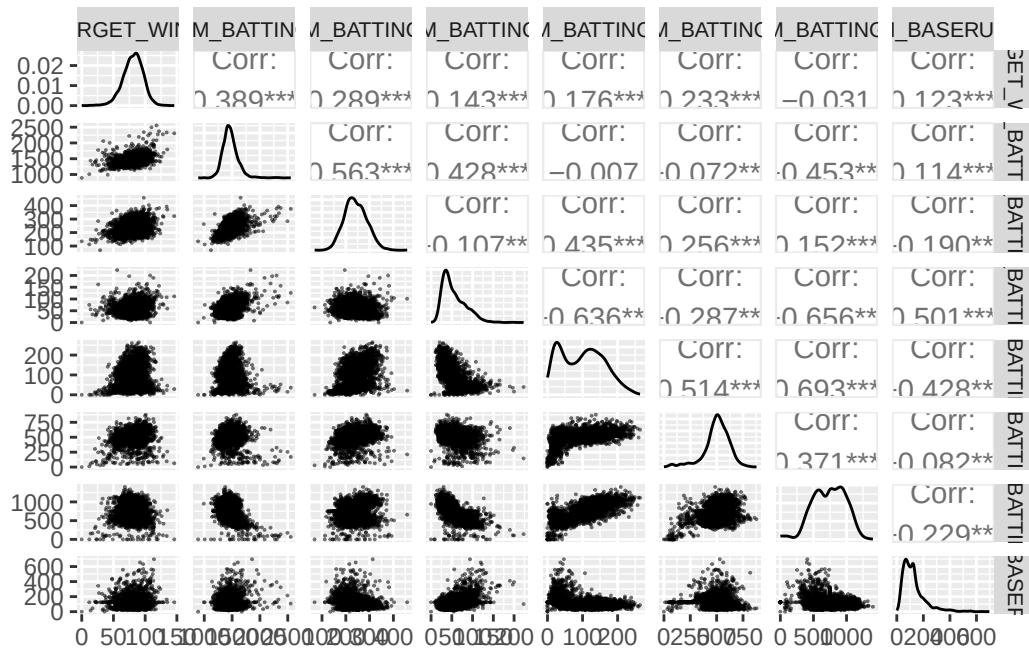
1. The independent and dependent variables are linearly related.
2. The errors are independent of each other.
3. The errors have constant variance.
4. The errors are normally distributed.

The last three will be evaluated after models are built (section 3). Here, we test the assumption of linearity using pair plots of the variables in the cleaned training dataset. To avoid overcrowded plots, we divided the variables into three groups.

2.3.1 Plot 1. Batting and baserun variables

- In the first row, the correlation coefficients show that all batting and baserun variables except TEAM_BATTING_SO have significant correlations with the target variable.

- ```
This block takes a few seconds
moneyball_clean |>
 select(-c(ends_with('MISSING')))) |> # omit categorical (flag) variables
 select(TARGET_WINS, starts_with('TEAM_BATTING'), TEAM_BASERUN_SB) |>
 ggpairs(
 # Color transparency and small data points help reduce overplotting
 mapping = aes(alpha = 0.1),
 lower = list(continuous = wrap('points', size = 0.01)),
 progress = FALSE
) +
 theme(
 strip.text = element_text(size = 8)
)
```



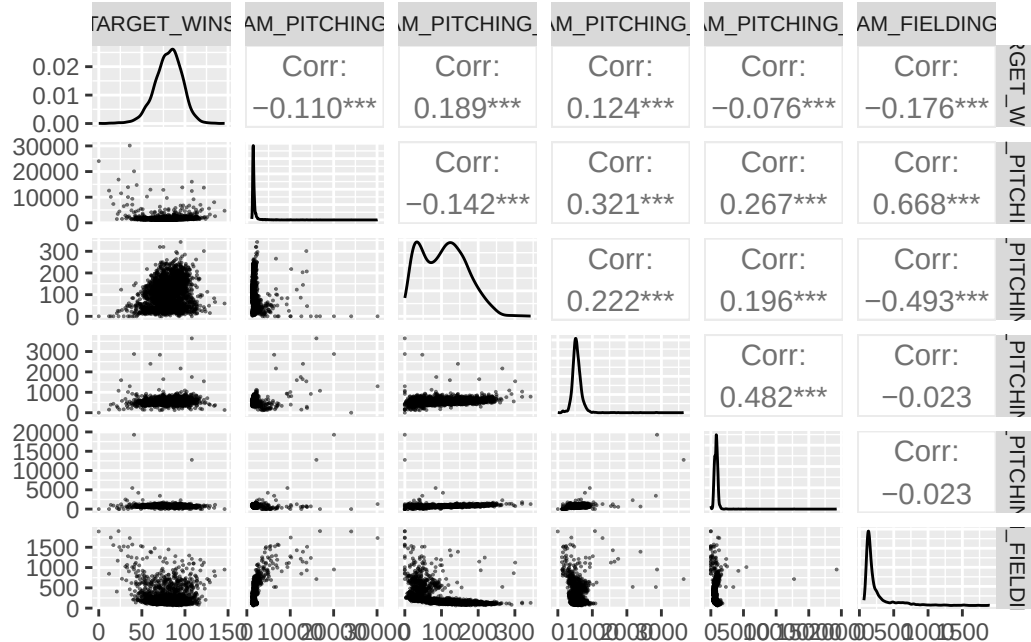
- In the first row, the correlation coefficients show that all pitching variables and the fielding variable have significant correlations with the target variable.
- In the first column, the scatterplots show that `TEAM_PITCHING_BB` has the clearest linear



relationship with `TARGET_WINS`.

- Two independent variables (`TEAM_PITCHING_H` and `TEAM_PITCHING_SO`) appear to be related by a horizontal line. Technically, this satisfies the assumption of linearity, but it suggests that these variables do not have predictive value.
- There are no obvious nonlinear relationships between pitching and fielding variables and the target variable.

```
This block takes a few seconds
moneyball_clean |>
 select(-c(ends_with('MISSING'))) |> # omit categorical (flag) variables
 select(TARGET_WINS, starts_with('TEAM_PITCHING'), TEAM_FIELDING_E) |>
 ggpairs(
 mapping = aes(alpha = 0.1),
 lower = list(continuous = wrap('points', size = 0.01)),
 progress = FALSE
) +
 theme(
 strip.text = element_text(size = 8)
)
```



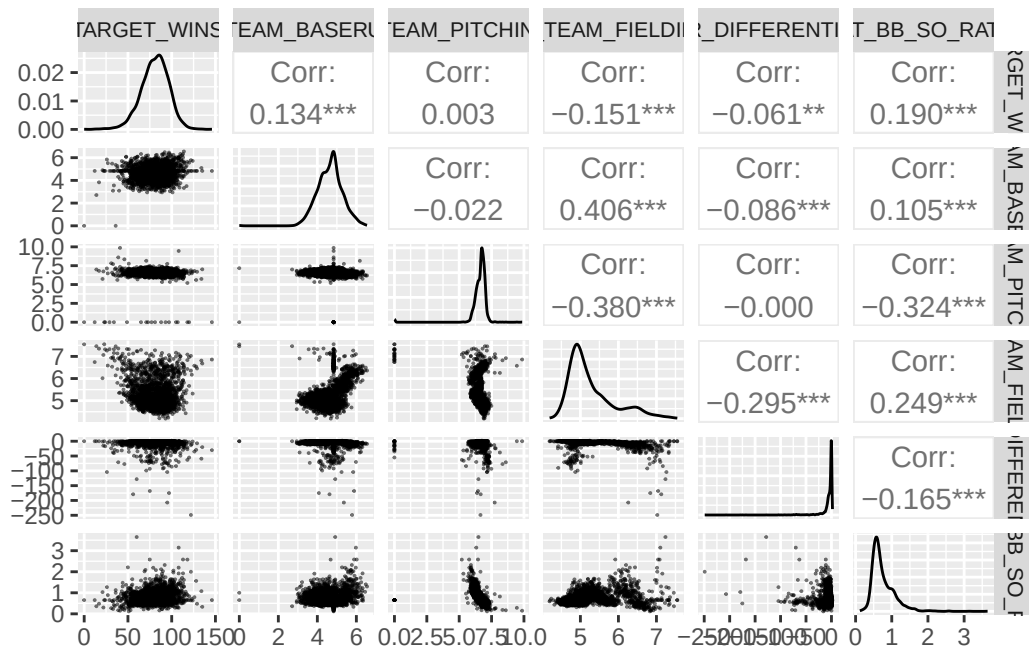
### 2.3.3 Plot 3. New variables

- In the first row, the correlation coefficients show that four of the five newly created variables have significant correlations with the target variable. The exception was

LOG\_TEAM\_PITCHING\_SO, which had a correlation coefficient of 0.003.

- In the first column, the scatterplots show that BAT\_BB\_SO\_RATIO has the clearest linear relationship with TARGET\_WINS.
  - LOG\_TEAM\_PITCHING\_SO appears to be related by a horizontal line. Although this satisfies the assumption of linearity, it suggests that this variable may not have predictive value.
- There are no obvious nonlinear relationships between the newly created variables and the target variable.

```
This block takes a few seconds
Pair plots for new variables
moneyball_clean |>
 select(TARGET_WINS, starts_with('LOG'), HR_DIFFERENTIAL, BAT_BB_SO_RATIO) |>
 ggpairs(
 mapping = aes(alpha = 0.1),
 lower = list(continuous = wrap('points', size = 0.01)),
 progress = FALSE
) +
 theme(
 strip.text = element_text(size = 8)
)
```



### 2.3.4 Conclusions

The results above indicate that the assumption of linearity is satisfied for all independent variables. However, several were flagged as being unlikely to have predictive value in a multivariate linear model.

## 2.4 Build Models

*Using the training data set, build at least three different multiple linear regression models, using different variables (or the same variables with different transformations). Since we have not yet covered automated variable selection methods, you should select the variables manually (unless you previously learned Forward or Stepwise selection, etc.). Since you manually selected a variable for inclusion into the model or exclusion into the model, indicate why this was done. Discuss the coefficients in the models, do they make sense? For example, if a team hits a lot of Home Runs, it would be reasonably expected that such a team would win more games. However, if the coefficient is negative (suggesting that the team would lose more games), then that needs to be discussed. Are you keeping the model even though it is counter intuitive? Why? The boss needs to know.*

## 2.5 Select Models

*Decide on the criteria for selecting the best multiple linear regression model. Will you select a model with slightly worse performance if it makes more sense or is more parsimonious? Discuss why you selected your model. For the multiple linear regression model, will you use a metric such as Adjusted  $R^2$ , RMSE, etc.? Be sure to explain how you can make inferences from the model, discuss multi-collinearity issues (if any), and discuss other relevant model output. Using the training data set, evaluate the multiple linear regression model based on (a) mean squared error, (b)  $R^2$ , (c)  $F$ -statistic, and (d) residual plots. Make predictions using the evaluation data set.*

## 3 Results

We predict that...

## 4 Conclusion

In conclusion...

## 5 Appendix: R Code

### 5.1 1. Data Exploration

First, we preview the data.

```
moneyball_train |> head()
```

```
A tibble: 6 x 17
 INDEX TARGET_WINS TEAM_BATTING_H TEAM_BATTING_2B TEAM_BATTING_3B
 <dbl> <dbl> <dbl> <dbl> <dbl>
1 1 39 1445 194 39
2 2 70 1339 219 22
3 3 86 1377 232 35
4 4 70 1387 209 38
5 5 82 1297 186 27
6 6 75 1279 200 36
i 12 more variables: TEAM_BATTING_HR <dbl>, TEAM_BATTING_BB <dbl>,
TEAM_BATTING_SO <dbl>, TEAM_BASERUN_SB <dbl>, TEAM_BASERUN_CS <dbl>,
TEAM_BATTING_HBP <dbl>, TEAM_PITCHING_H <dbl>, TEAM_PITCHING_HR <dbl>,
TEAM_PITCHING_BB <dbl>, TEAM_PITCHING_SO <dbl>, TEAM_FIELDING_E <dbl>,
TEAM_FIELDING_DP <dbl>
```

The data are all numeric, and the target variable is `TARGET_WINS`, which represents the number of wins for the team in that season. The other variables represent various performance statistics for the team.

We also check summary statistics for all variables in our training data, including checking missingness.

```
moneyball_train |> skim()
```

Table 3: Data summary

|                        |                 |
|------------------------|-----------------|
| Name                   | moneyball_train |
| Number of rows         | 2276            |
| Number of columns      | 17              |
| Column type frequency: |                 |
| numeric                | 17              |
| Group variables        | None            |

### Variable type: numeric

| skim_vari-<br>able | n_miss-<br>ing | com-<br>plete_rate | mean    | sd     | p0 | p25    | p50    | p75     | p100 | hist |
|--------------------|----------------|--------------------|---------|--------|----|--------|--------|---------|------|------|
| INDEX              | 0              | 1.00               | 1268.46 | 736.35 | 1  | 630.75 | 1270.5 | 1915.50 | 2535 |      |

| skim_vari-<br>able    | n_miss-<br>ing | com-<br>plete_rate | mean    | sd      | p0   | p25     | p50    | p75     | p100  | hist |
|-----------------------|----------------|--------------------|---------|---------|------|---------|--------|---------|-------|------|
| TAR-<br>GET_WINS      | 0              | 1.00               | 80.79   | 15.75   | 0    | 71.00   | 82.0   | 92.00   | 146   |      |
| TEAM_BAT-<br>TING_H   | 0              | 1.00               | 1469.27 | 144.59  | 891  | 1383.00 | 1454.0 | 1537.25 | 2554  |      |
| TEAM_BAT-<br>TING_2B  | 0              | 1.00               | 241.25  | 46.80   | 69   | 208.00  | 238.0  | 273.00  | 458   |      |
| TEAM_BAT-<br>TING_3B  | 0              | 1.00               | 55.25   | 27.94   | 0    | 34.00   | 47.0   | 72.00   | 223   |      |
| TEAM_BAT-<br>TING_HR  | 0              | 1.00               | 99.61   | 60.55   | 0    | 42.00   | 102.0  | 147.00  | 264   |      |
| TEAM_BAT-<br>TING_BB  | 0              | 1.00               | 501.56  | 122.67  | 0    | 451.00  | 512.0  | 580.00  | 878   |      |
| TEAM_BAT-<br>TING_SO  | 102            | 0.96               | 735.61  | 248.53  | 0    | 548.00  | 750.0  | 930.00  | 1399  |      |
| TEAM_BASERUN1B        | 138            | 0.94               | 124.76  | 87.79   | 0    | 66.00   | 101.0  | 156.00  | 697   |      |
| TEAM_BASERUN2B        | 772            | 0.66               | 52.80   | 22.96   | 0    | 38.00   | 49.0   | 62.00   | 201   |      |
| TEAM_BAT-<br>TING_HBP | 2085           | 0.08               | 59.36   | 12.97   | 29   | 50.50   | 58.0   | 67.00   | 95    |      |
| TEAM_PITCH-<br>ING_H  | 0              | 1.00               | 1779.21 | 1406.84 | 1137 | 1419.00 | 1518.0 | 1682.50 | 30132 |      |
| TEAM_PITCH-<br>ING_HR | 0              | 1.00               | 105.70  | 61.30   | 0    | 50.00   | 107.0  | 150.00  | 343   |      |
| TEAM_PITCH-<br>ING_BB | 0              | 1.00               | 553.01  | 166.36  | 0    | 476.00  | 536.5  | 611.00  | 3645  |      |
| TEAM_PITCH-<br>ING_SO | 102            | 0.96               | 817.73  | 553.09  | 0    | 615.00  | 813.5  | 968.00  | 19278 |      |
| TEAM_FIELD-<br>ING_E  | 0              | 1.00               | 246.48  | 227.77  | 65   | 127.00  | 159.0  | 249.25  | 1898  |      |
| TEAM_FIELD-<br>ING_DP | 286            | 0.87               | 146.39  | 26.23   | 52   | 131.00  | 149.0  | 164.00  | 228   |      |

```
nrows <- moneyball_train |> nrow()
ncols <- moneyball_train |> length()
perc_complete <- round(moneyball_train |> n_complete() / nrows / ncols * 100, 2)
```

This training dataset contains 2276 rows and 17 features, and is 91.01 percent complete. The variables with missingness are:

```
moneyball_train |>
 summarise(across(everything(), ~ sum(is.na(.)) / nrow * 100)) |>
 pivot_longer(everything(), names_to = "variable", values_to = "missing_percentage") |>
 arrange(desc(missing_percentage)) |>
 filter(missing_percentage > 0)
```

# A tibble: 6 x 2

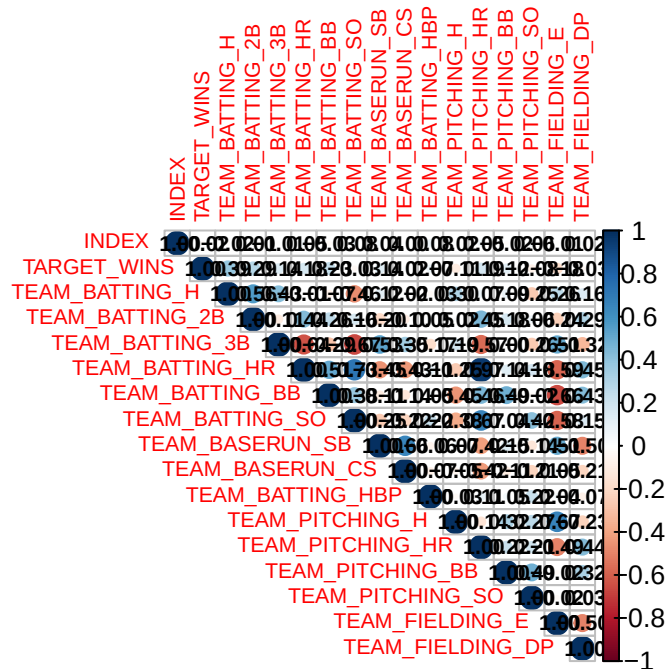
|   | variable         | missing_percentage |
|---|------------------|--------------------|
|   | <chr>            | <dbl>              |
| 1 | TEAM_BATTING_HBP | 91.6               |
| 2 | TEAM_BASERUN_CS  | 33.9               |
| 3 | TEAM_FIELDING_DP | 12.6               |
| 4 | TEAM_BASERUN_SB  | 5.76               |
| 5 | TEAM_BATTING_SO  | 4.48               |
| 6 | TEAM_PITCHING_SO | 4.48               |

This missingness will need to be addressed in the data preparation step.

We also visualize correlations between all variables.

```
moneyball_train_cor <- moneyball_train |>
 cor(use = "pairwise.complete.obs")

corrplot(moneyball_train_cor, method = "circle", type = "upper", tl.cex = 0.7, sig.level = 0.05)
```



The most correlated variables with the target variable TARGET\_WINS are:

```
moneyball_train_cor |>
 as.data.frame() |>
 rownames_to_column(var = "variable") |>
 select(variable, TARGET_WINS) |>
 filter(variable != "TARGET_WINS") |>
 arrange(desc(abs(TARGET_WINS))) |>
 head(10)
```

|    | variable         | TARGET_WINS |
|----|------------------|-------------|
| 1  | TEAM_BATTING_H   | 0.3887675   |
| 2  | TEAM_BATTING_2B  | 0.2891036   |
| 3  | TEAM_BATTING_BB  | 0.2325599   |
| 4  | TEAM_PITCHING_HR | 0.1890137   |
| 5  | TEAM_FIELDING_E  | -0.1764848  |
| 6  | TEAM_BATTING_HR  | 0.1761532   |
| 7  | TEAM_BATTING_3B  | 0.1426084   |
| 8  | TEAM_BASERUN_SB  | 0.1351389   |
| 9  | TEAM_PITCHING_BB | 0.1241745   |
| 10 | TEAM_PITCHING_H  | -0.1099371  |

These may be strong predictors of wins.

The most correlated independent variables are:

```
moneyball_train_cor |>
 as.data.frame() |>
 rownames_to_column(var = "variable1") |>
 pivot_longer(-variable1, names_to = "variable2", values_to = "correlation") |>
 filter(variable1 != variable2) |>
 distinct(across(c(correlation)), .keep_all = TRUE) |>
 arrange(desc(abs(correlation))) |>
 head(20)
```

# A tibble: 20 x 3

|   | variable1       | variable2        | correlation |
|---|-----------------|------------------|-------------|
|   | <chr>           | <chr>            | <dbl>       |
| 1 | TEAM_BATTING_HR | TEAM_PITCHING_HR | 0.969       |
| 2 | TEAM_BATTING_HR | TEAM_BATTING_SO  | 0.727       |
| 3 | TEAM_BATTING_3B | TEAM_BATTING_SO  | -0.670      |
| 4 | TEAM_PITCHING_H | TEAM_FIELDING_E  | 0.668       |
| 5 | TEAM_BATTING_SO | TEAM_PITCHING_HR | 0.667       |
| 6 | TEAM_BATTING_BB | TEAM_FIELDING_E  | -0.656      |

|    |                  |                  |        |
|----|------------------|------------------|--------|
| 7  | TEAM_BASERUN_SB  | TEAM_BASERUN_CS  | 0.655  |
| 8  | TEAM_BATTING_3B  | TEAM_BATTING_HR  | -0.636 |
| 9  | TEAM_BATTING_HR  | TEAM_FIELDING_E  | -0.587 |
| 10 | TEAM_BATTING_SO  | TEAM_FIELDING_E  | -0.585 |
| 11 | TEAM_BATTING_3B  | TEAM_PITCHING_HR | -0.568 |
| 12 | TEAM_BATTING_H   | TEAM_BATTING_2B  | 0.563  |
| 13 | TEAM_BATTING_3B  | TEAM_BASERUN_SB  | 0.534  |
| 14 | TEAM_BATTING_HR  | TEAM_BATTING_BB  | 0.514  |
| 15 | TEAM_BATTING_3B  | TEAM_FIELDING_E  | 0.510  |
| 16 | TEAM_BASERUN_SB  | TEAM_FIELDING_E  | 0.510  |
| 17 | TEAM_FIELDING_E  | TEAM_FIELDING_DP | -0.498 |
| 18 | TEAM_BASERUN_SB  | TEAM_FIELDING_DP | -0.497 |
| 19 | TEAM_PITCHING_HR | TEAM_FIELDING_E  | -0.493 |
| 20 | TEAM_BATTING_BB  | TEAM_PITCHING_BB | 0.489  |

It makes sense that homeruns by batters (TEAM\_BATTING\_HR) and homeruns allowed (TEAM\_PITCHING\_HR) are correlated, as teams that hit many homeruns may also allow many homeruns, for example. We may need to avoid including these pairs of variables in the same model to prevent multicollinearity.

We also visualize the distribution of z-scores of all variable observations, and identify outliers in the training data.

```
moneyball_train |>
 pivot_longer(cols = everything(), names_to = "Variable", values_to = "Value") |>
 group_by(Variable) |>
 mutate(Z_Score = scale(Value)[,1]) |>
 ungroup() |>
 ggplot(aes(x = Z_Score, y = Variable)) +
 geom_boxplot() +
 theme_minimal() +
 theme(panel.grid = element_blank(), axis.text.y = element_text(angle = 0)) +
 xlab("Z-Score")
```

Warning: Removed 3478 rows containing non-finite outside the scale range  
(`stat\_boxplot()`).



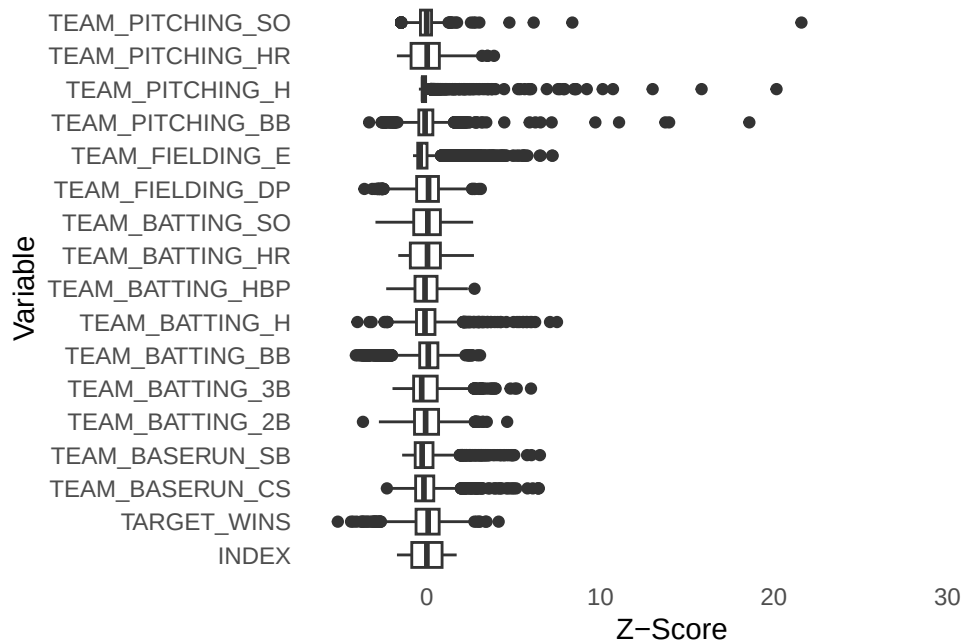
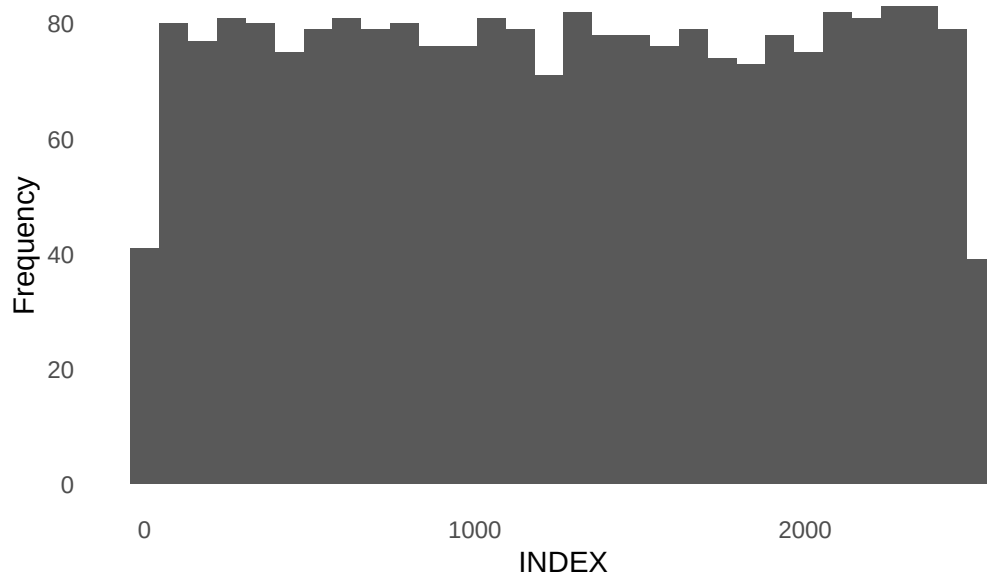


Figure 4: Box plots for identifying outliers in the training data

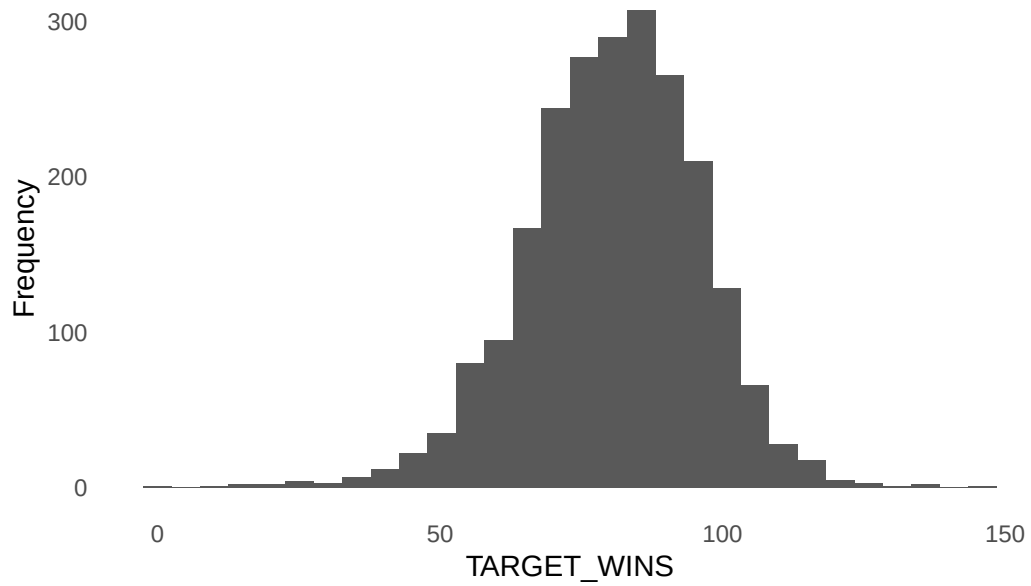
We also visualize the distribution of all variables.

```
Loop through all variables and create histograms
for (var in names(moneyball_train)) {
 p <- ggplot(moneyball_train, aes_string(x = var)) +
 geom_histogram() +
 labs(title = paste("Distribution of", var), x = var, y = "Frequency") +
 theme_minimal() +
 theme(panel.grid = element_blank())
 print(p)
}
```

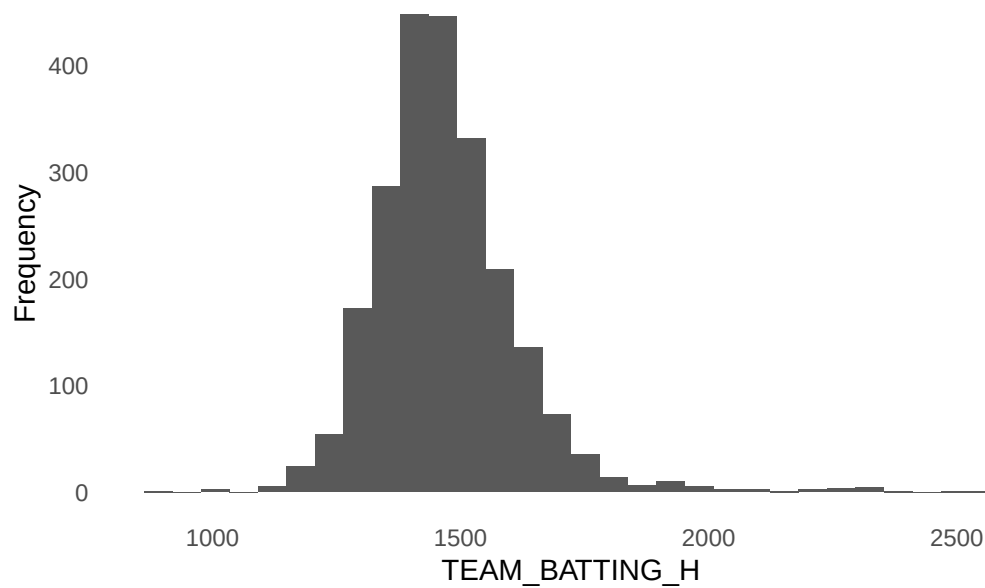
Distribution of INDEX



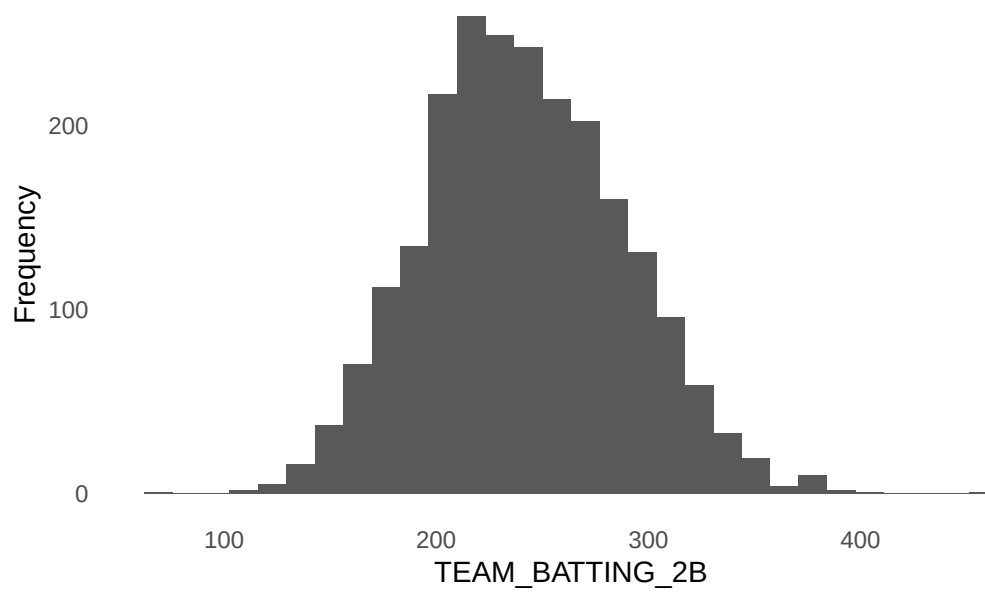
Distribution of TARGET\_WINS

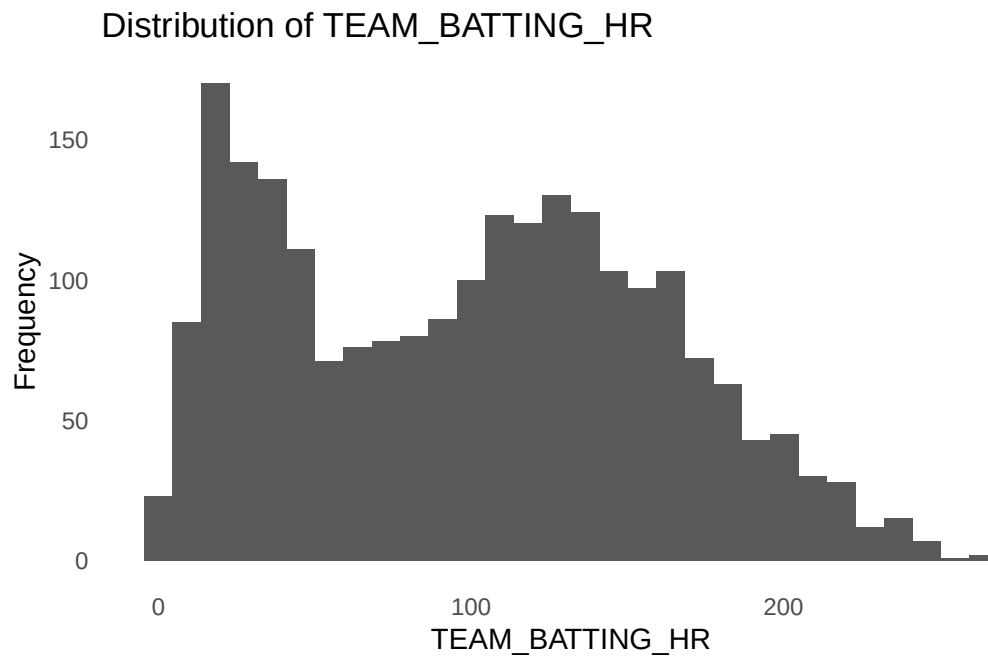
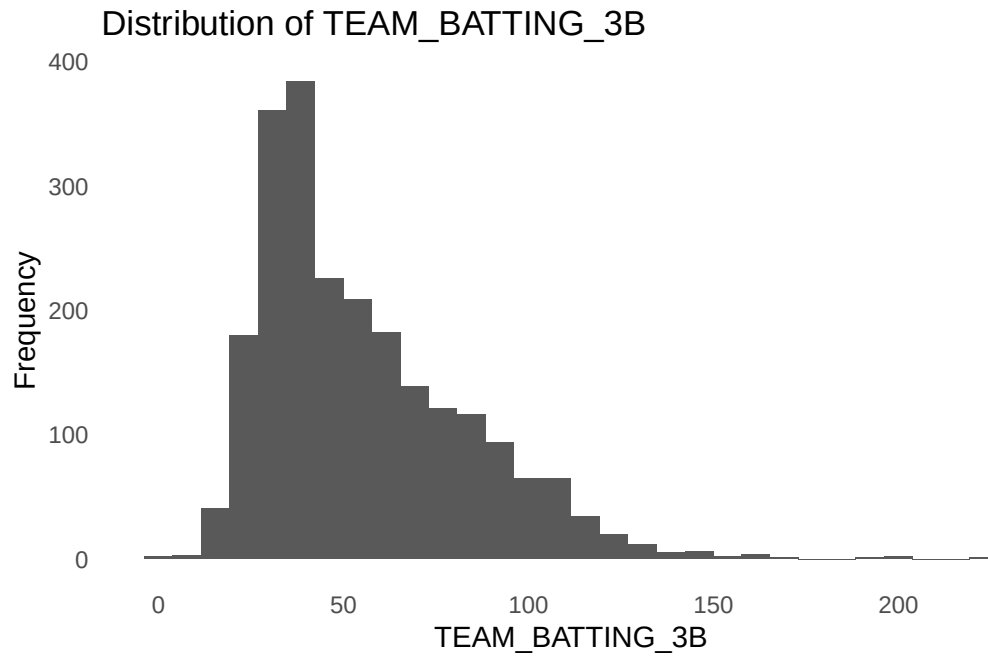


Distribution of TEAM\_BATTING\_H

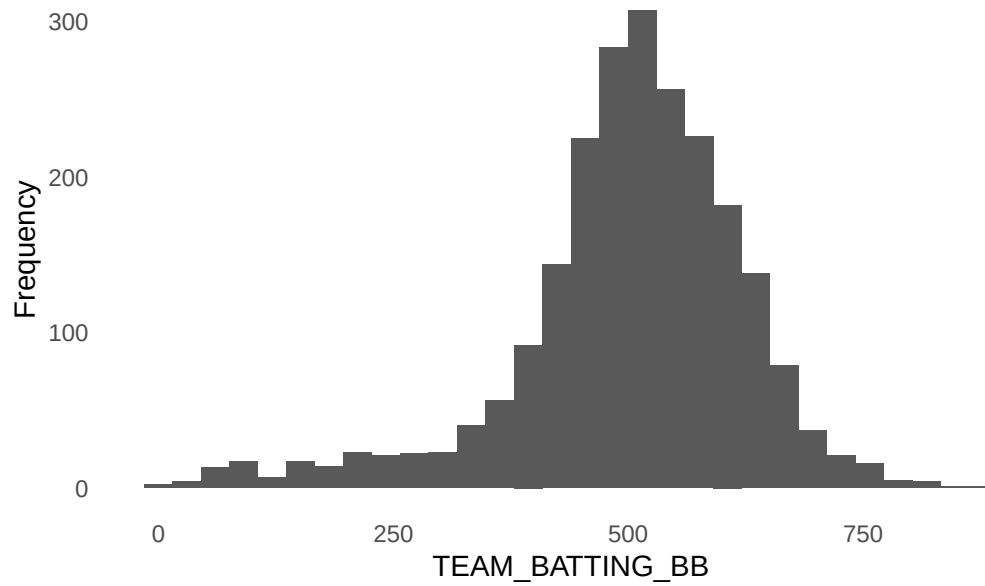


Distribution of TEAM\_BATTING\_2B

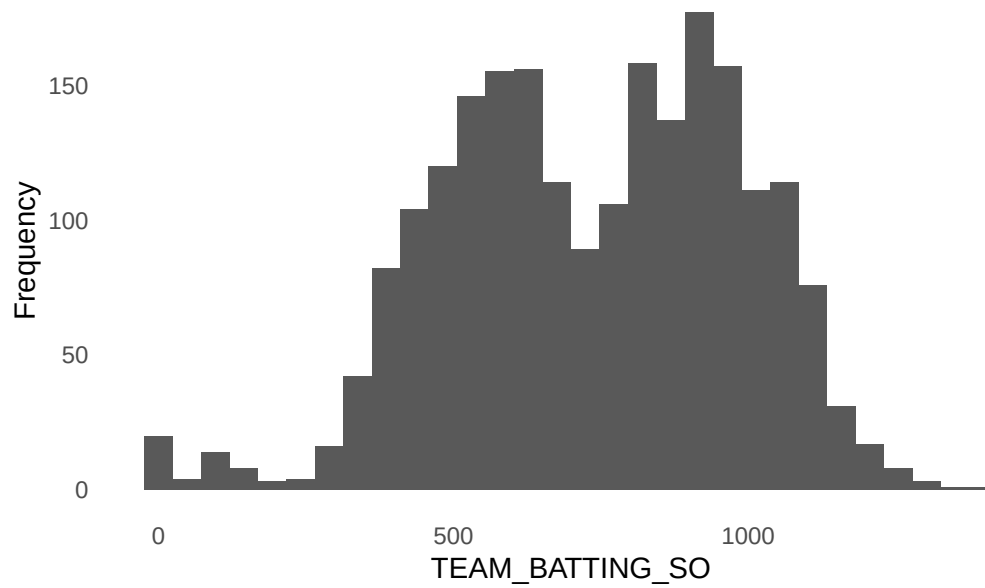




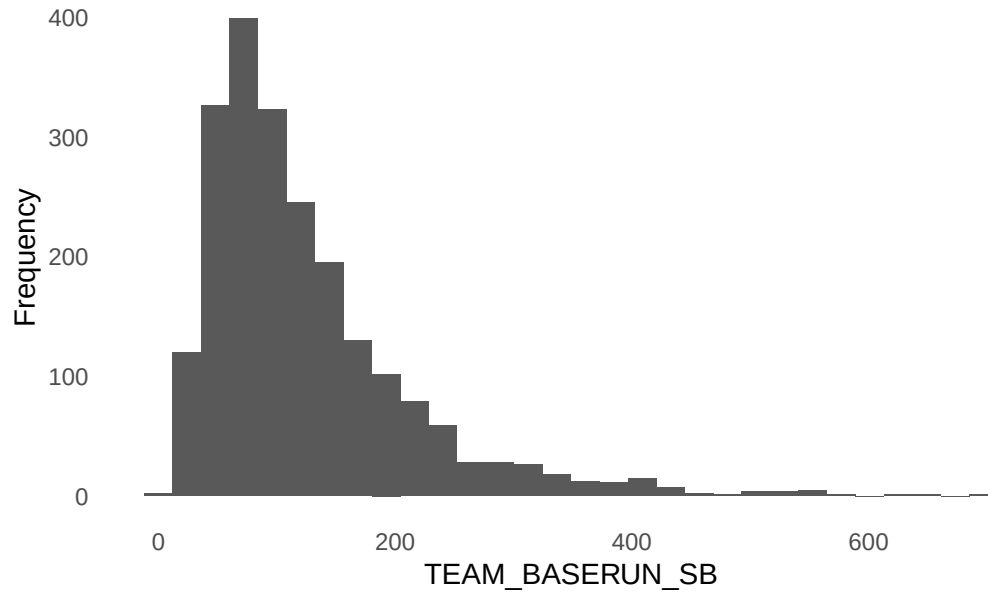
Distribution of TEAM\_BATTING\_BB



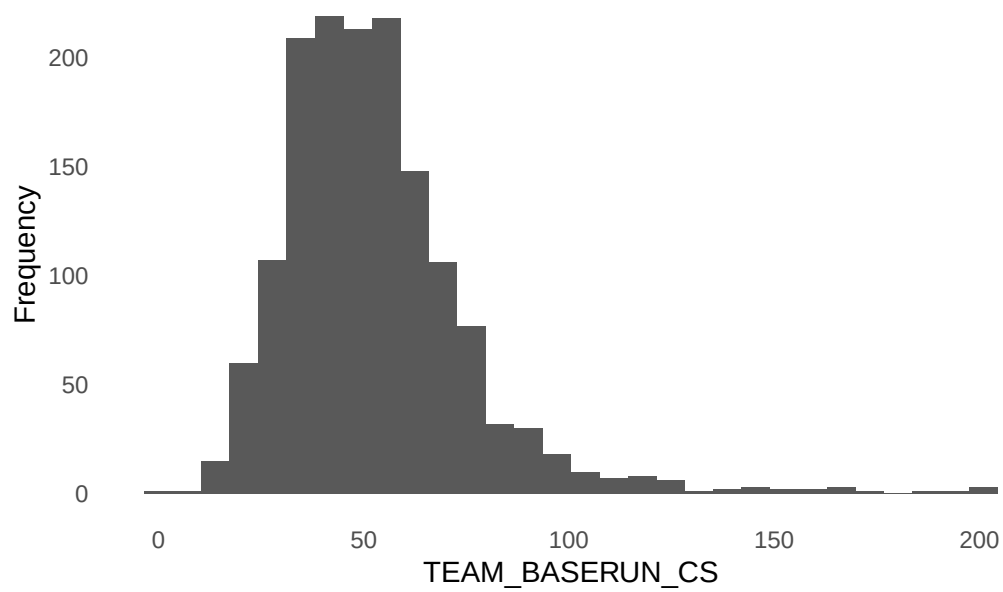
Distribution of TEAM\_BATTING\_SO

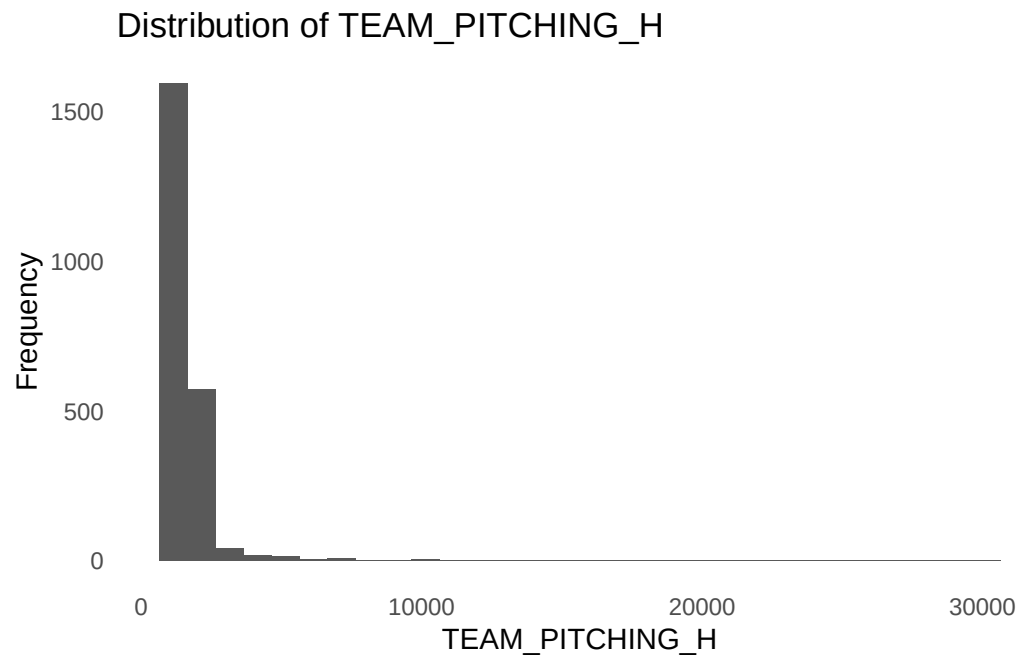
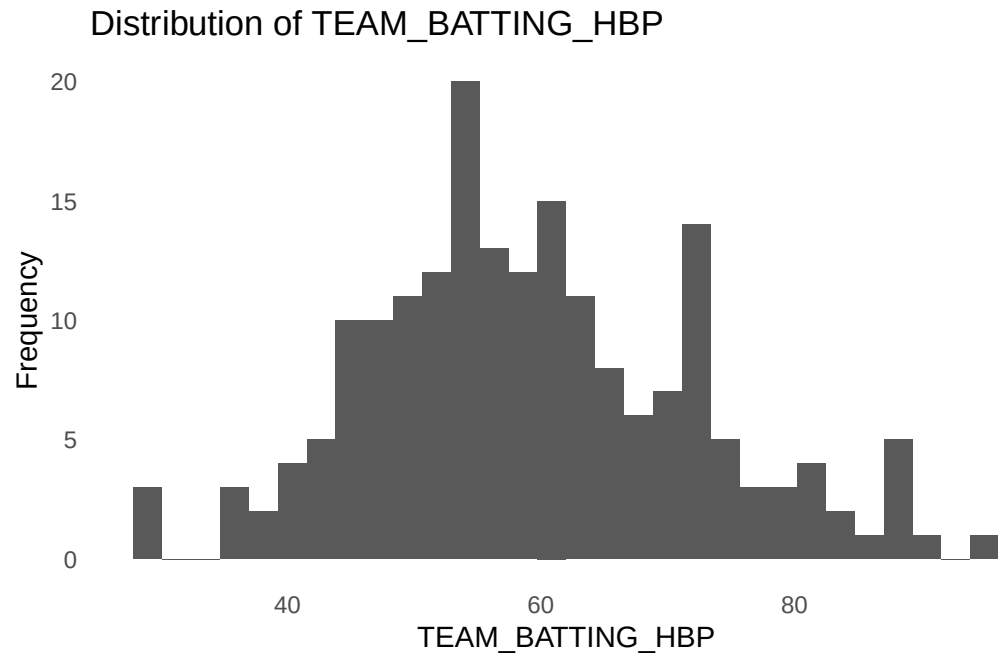


Distribution of TEAM\_BASERUN\_SB

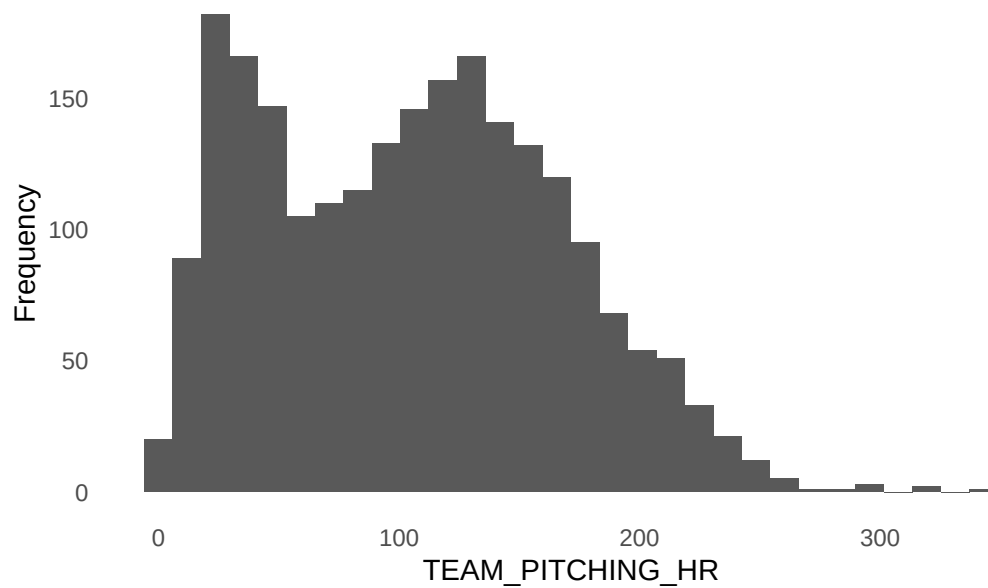


Distribution of TEAM\_BASERUN\_CS

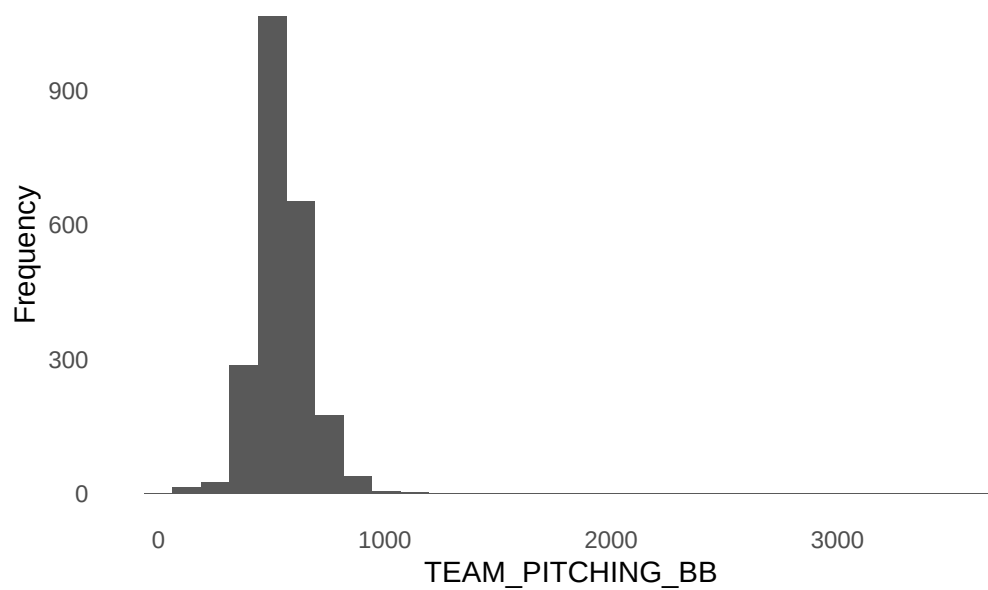




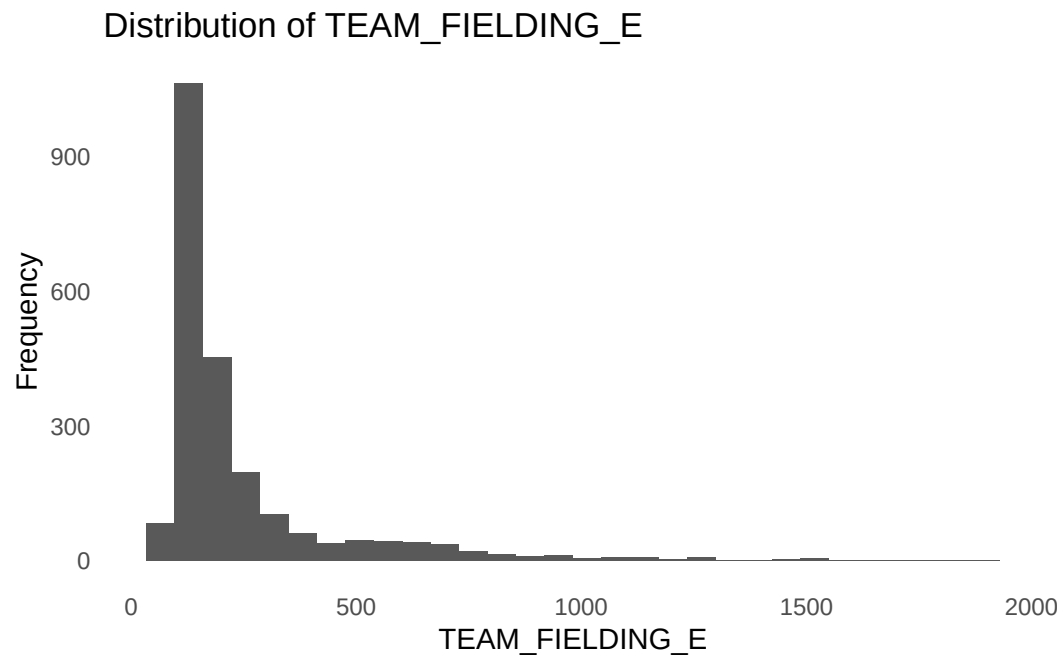
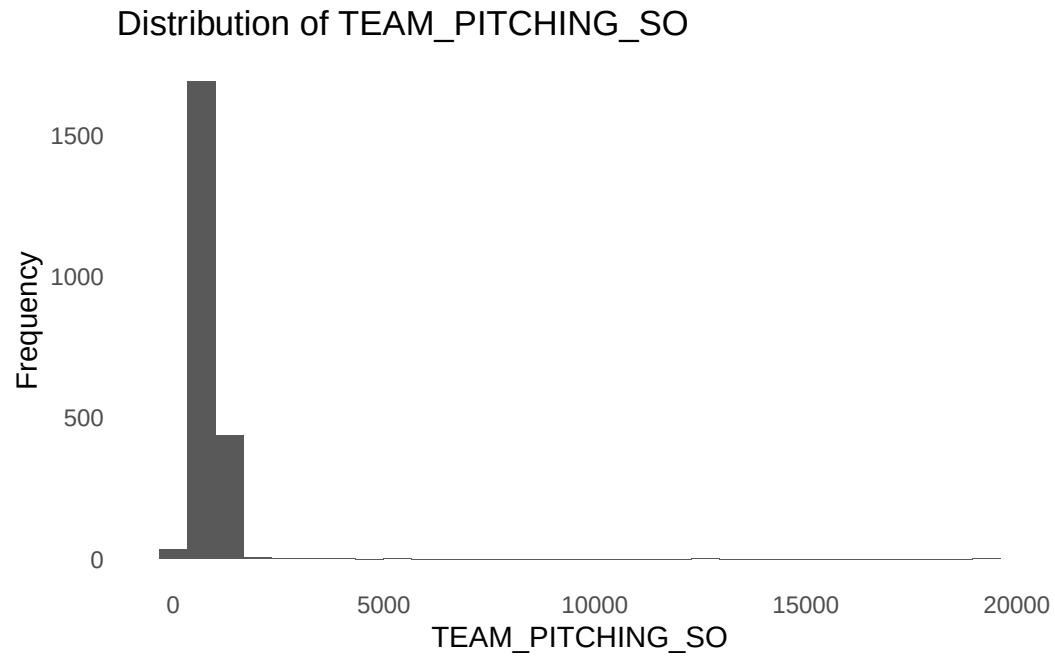
Distribution of TEAM\_PITCHING\_HR

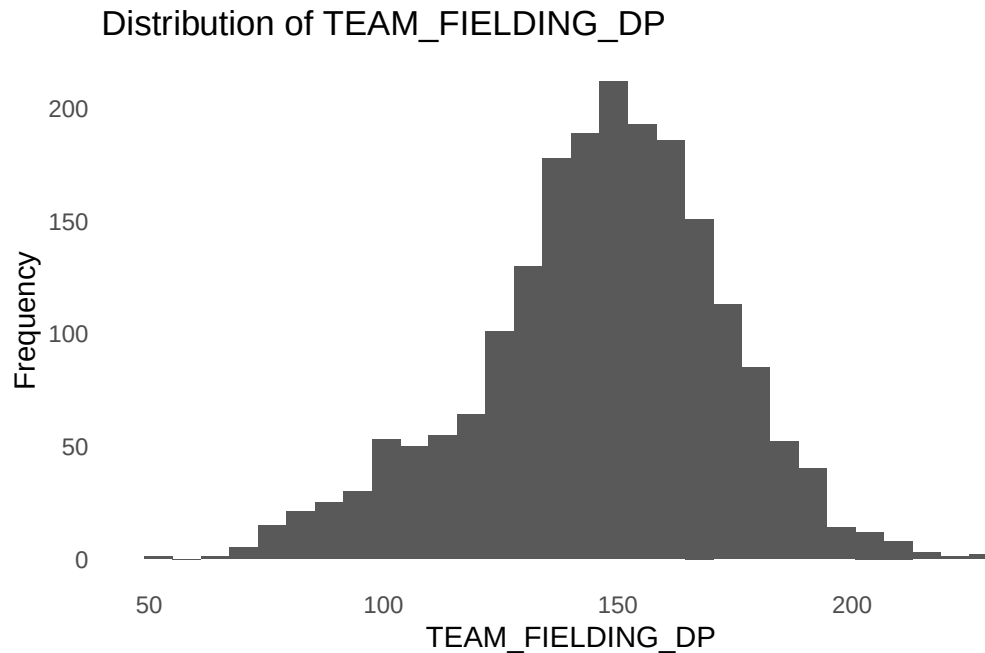


Distribution of TEAM\_PITCHING\_BB









## 5.2 2. Data Preparation

Next, we will prepare the data for modeling.

First, we handle missing data. Since we saw in the EDA that `TEAM_BATTING_HBP`, `TEAM_BASERUN_CS`, and `TEAM_FIELDING_DP` were 91.6%, 33.9%, and 12.6% missing, respectively, and not very correlated with the target variable, so we will drop these variables. The remaining variables are less than 10% missing, so we will impute the missing values with the median of each variable. `TEAM_BASERUN_SB` is the most correlated variable with the target variable that has missingness, so we will be sure to impute this variable rather than drop it.

The median imputation was picked based on the distributions of the variables with missingness: `TEAM_BASERUN_SB` and `TEAM_PITCHING_SO` have a long right tail so the median is a better imputation method than the mean. `TEAM_BATTING_SO` is bimodal, so the median is also a better imputation method than the mean for this variable. See the histograms above for the distributions of these variables.

```
moneyball_train_prepared <- moneyball_train |>
 select(-TEAM_BATTING_HBP, -TEAM_BASERUN_CS, -TEAM_FIELDING_DP) |>
 mutate(across(everything(), ~ ifelse(is.na(.), median(., na.rm = TRUE), .)))
```

### **5.3 3. Build Models**

### **5.4 4. Select Models**

Export assigned predictions (the number of wins for the team) for the evaluation data set for deliverable.